

AUTO HEADZ

Full Stack Automotive Website Project Report

Internship / Academic Presentation Report

Submitted By

Project Name: Auto Headz

Project Type: Full Stack Automotive Web Application

Technologies Used: HTML, CSS, JavaScript, Python Flask, PostgreSQL

1. Introduction

Auto Headz is a modern full-stack automotive web application developed as an internship and academic project. The website allows users to explore various car brands, view detailed car specifications, search vehicles, and interact with dynamic backend features.

The project was initially developed using SQLite as the database system. Later, the application architecture was upgraded to PostgreSQL to improve scalability, reliability, and production-level database management.

This project demonstrates frontend development, backend API integration, authentication systems, database management, and database migration.

2. Project Objectives

The main objectives of this project are:

1. Build a responsive automotive website.
2. Create a backend system using Flask.
3. Store car information dynamically in a database.
4. Implement APIs for data communication.
5. Add advanced database functionality.
6. Upgrade the project from SQLite to PostgreSQL.
7. Make the project suitable for real-world deployment.
8. Improve understanding of full-stack development.

3. Technology Stack

Frontend Technologies

- HTML5
- CSS3
- JavaScript

Backend Technologies

- Python
- Flask

Database Technologies

Initial Database

- SQLite

Upgraded Database

- PostgreSQL

Additional Tools

- SQLAlchemy ORM
 - JWT Authentication
 - pgAdmin 4
 - Git Bash
 - OpenCode AI Development Tool
-

4. System Architecture

The project follows a full-stack architecture:

1. Frontend sends requests.
2. Backend APIs process requests.
3. Database stores and retrieves data.
4. Backend returns responses to frontend.
5. Frontend dynamically updates the website.

Workflow

Browser → Flask Backend → PostgreSQL Database → Backend Response → Frontend Display

5. Initial Database System (SQLite)

At the beginning of the project, SQLite was used.

Why SQLite Was Used Initially

1. Easy setup.
2. Lightweight database.
3. Good for development and testing.
4. No separate database server required.

How SQLite Worked

SQLite stored all data inside a single .db file.

Example:

```
sqlite:///cars.db
```

Whenever users interacted with the website:

- Car data was stored in the database file.
- Reviews were saved in the same file.
- Authentication information was also stored.

Although SQLite was simple and effective, it had limitations for larger applications.

6. Need for PostgreSQL Migration

As the project expanded, PostgreSQL became necessary.

Problems with SQLite

1. Limited scalability.
2. Not ideal for multiple users.
3. Less suitable for production environments.
4. Limited advanced database features.
5. File-based storage system.

Advantages of PostgreSQL

1. Professional database management system.
2. Better performance.
3. Supports multiple users.
4. More secure.
5. Better for cloud deployment.
6. Advanced querying capabilities.
7. Industry-standard database system.

Therefore, the database system was migrated from SQLite to PostgreSQL.

7. PostgreSQL Installation Process

The PostgreSQL database server was installed successfully.

Installation Steps

1. Downloaded PostgreSQL installer.
2. Installed PostgreSQL Server.
3. Installed pgAdmin 4.
4. Configured database password.
5. Kept default port number 5432.
6. Created a new PostgreSQL database named:

carshowroom

8. PostgreSQL Migration Process

The website backend was successfully migrated from SQLite to PostgreSQL.

Migration Process Overview

Step 1: SQLite Configuration Removed

Old SQLite database configuration was removed.

Example:

sqlite:///cars.db

Step 2: PostgreSQL Connection Added

The backend was configured to connect with PostgreSQL.

Example:

```
postgresql://postgres:password@localhost:5432/carshowroom
```

Step 3: PostgreSQL Packages Installed

Required PostgreSQL drivers and packages were installed.

Example:

```
pip install psycopg2-binary
```

Step 4: Database Models Updated

SQLAlchemy models were connected to PostgreSQL.

Step 5: Tables Migrated

All application tables were migrated successfully.

Tables include:

- Users
- Cars
- Reviews

Step 6: Backend Successfully Connected

The Flask backend connected successfully to PostgreSQL.

9. How Data Is Stored in PostgreSQL

Unlike SQLite, PostgreSQL stores data inside a database server.

Data Storage Process

When a user adds a car:

1. Frontend sends request.
2. Flask backend receives request.
3. SQLAlchemy converts data into SQL query.
4. PostgreSQL stores data in tables.
5. Backend sends response to frontend.

Example Data Storage Flow

Frontend Form → Flask API → SQLAlchemy ORM → PostgreSQL Database

10. Database Tables

Users Table

Stores:

- User ID
- Username
- Email
- Password Hash
- User Role

Cars Table

Stores:

- Car Name
- Brand
- Engine Details
- Horsepower
- Mileage
- Price
- Images
- Official Links

Reviews Table

Stores:

- Review ID
 - User ID
 - Car ID
 - Rating
 - Comments
-

11. Backend Functionality

The backend was developed using Flask.

Main Backend Features

1. REST API creation.
 2. Database communication.
 3. Authentication system.
 4. CRUD operations.
 5. Admin management.
 6. Review management.
 7. Data validation.
-

12. Authentication System

JWT (JSON Web Token) authentication was implemented.

Features

1. User registration.
 2. User login.
 3. Token generation.
 4. Protected routes.
 5. Admin role verification.
-

13. API Endpoints

Authentication APIs

POST /api/auth/register
POST /api/auth/login
GET /api/auth/me

Cars APIs

GET /api/cars
GET /api/cars/<id>
POST /api/cars
PUT /api/cars/<id>
DELETE /api/cars/<id>

Reviews APIs

```
GET    /api/reviews
POST   /api/reviews
DELETE /api/reviews/<id>
```

14. Frontend Features

The frontend was designed to provide an attractive automotive experience.

Main Features

1. Brand sections.
 2. Car cards.
 3. Search functionality.
 4. Responsive design.
 5. Animations.
 6. Car specifications.
 7. Brand website links.
 8. Embedded videos.
 9. Dynamic data loading.
-

15. Daily Project Execution

To run the project locally, two terminals are used.

Backend Terminal

```
py -m backend.app
```

This starts:

- Flask backend
 - APIs
 - PostgreSQL connection
-

Frontend Terminal

```
npm run dev
```

This starts:

- Frontend development server
 - Website interface
-

16. Testing Performed

The following testing was performed:

Functional Testing

1. Car data loading.
2. Search functionality.
3. User login.
4. Review submission.
5. PostgreSQL data insertion.
6. API responses.
7. Admin operations.

Database Testing

1. Verified PostgreSQL connection.
 2. Verified table creation.
 3. Verified data persistence.
 4. Verified CRUD operations.
-

17. Results Achieved

The project successfully achieved:

1. Full-stack architecture.
 2. Dynamic backend integration.
 3. PostgreSQL database migration.
 4. Real-time database operations.
 5. API-driven system.
 6. Authentication system.
 7. Scalable database infrastructure.
 8. Production-ready backend setup.
-

18. Skills Learned During Development

Through this project, the following technical skills were learned:

1. Frontend development.
 2. Backend API development.
 3. Flask framework usage.
 4. Database management.
 5. PostgreSQL setup.
 6. SQLAlchemy ORM.
 7. Authentication systems.
 8. Database migration.
 9. Git Bash terminal usage.
 10. Full-stack architecture understanding.
-

19. Challenges Faced

During development, several challenges were encountered:

1. Database connection issues.
2. Backend configuration errors.
3. Frontend integration issues.
4. Search functionality debugging.
5. Migration from SQLite to PostgreSQL.
6. API routing problems.
7. Package installation and environment setup.

These issues were resolved through debugging, testing, and iterative development.

20. Future Improvements

Future improvements planned for the project include:

1. Advanced search filters.
2. AI-based car recommendations.
3. User profile system.
4. Password reset functionality.
5. Cloud deployment.
6. Real-time chat support.

7. Performance optimization.
 8. Mobile application version.
 9. Payment integration.
 10. Cars Buying selling Option
-

21. Conclusion

Auto Headz successfully evolved from a simple frontend website into a complete full-stack automotive platform.

The migration from SQLite to PostgreSQL significantly improved the project architecture and made the application more scalable and professional.

This project demonstrates practical knowledge in:

- Frontend development
- Backend development
- Database systems
- API integration
- Authentication systems
- Database migration
- Full-stack application architecture

The project also provided valuable real-world development experience and strengthened practical understanding of modern web technologies.

22. Final Outcome

The project now includes:

- ✓ Responsive frontend
 - ✓ Flask backend APIs
 - ✓ PostgreSQL database integration
 - ✓ Dynamic data storage
 - ✓ Authentication system
 - ✓ Review system
 - ✓ Admin operations
 - ✓ Production-ready database architecture
-

Thank You